

Projeto de funções

Programação Funcional

Marco A L Barbosa

malbarbo.pro.br

Departamento de Informática

Universidade Estadual de Maringá



Este trabalho está licenciado com uma Licença Creative Commons - Atribuição-Compartilhagual 4.0 Internacional.

<http://github.com/malbarbo/na-progfun>

0 processo

Vamos voltar ao problema da Márcia.

Depois que você fez o programa para o Alan, a Márcia, amiga em comum de vocês, soube que você está oferecendo serviços desse tipo e também quer a sua ajuda. O problema da Márcia é que ela sempre tem que fazer a conta manualmente para saber se deve abastecer o carro com álcool ou gasolina. A conta que ela faz é verificar se o preço do álcool é até 70% do preço da gasolina, se sim, ela abastece o carro com álcool, senão ela abastece o carro com gasolina. Você pode ajudar a Márcia também?

Como proceder para projetar este programa?

Vamos usar o mesmo processo de projeto de funções que vocês viram em Fundamentos de Algoritmos. O processo é o mesmo; o que muda é como especificar e implementar usando as técnicas de programação funcional.

Este capítulo é, portanto, uma revisão do processo

- Análise
- Definição dos tipos de dados
- Especificação
- Implementação
- Verificação
- Revisão

Esse processo é inspirado no livro [How to Design Programs](#).

Cada etapa tem um objetivo e depende das etapas anteriores

- Análise: identificar o problema a ser resolvido
- Definição dos tipos de dados: definir como as informações serão representadas
- Especificação: especificar com precisão o que a função deve fazer
- Implementação: implementar a função de acordo com a especificação
- Verificação: verificar se a implementação está de acordo com a especificação
- Revisão: identificar e fazer melhorias na especificação e implementação

Note que às vezes precisamos alterar a ordem das etapas, por exemplo, podemos estar na implementação e encontrar uma nova condição, e então devemos voltar e alterar a especificação; ou podemos escrever os exemplos antes de terminar o propósito, para entender melhor o problema.

Mas devemos evitar fazer a implementação diretamente!

Mas esse processo serve para projetar funções, como projetamos programas?

Um programa é composto de várias funções, então temos que decompor o programa em funções e aplicar o processo para projetar cada função.

Vamos treinar com problemas simples, de uma função, depois vamos utilizar o processo em problemas mais elaborados.

Exemplo - combustível

Depois que você fez o programa para o Alan, a Márcia, amiga em comum de vocês, soube que você está oferecendo serviços desse tipo e também quer a sua ajuda. O problema da Márcia é que ela sempre tem que fazer a conta manualmente para saber se deve abastecer o carro com álcool ou gasolina. A conta que ela faz é verificar se o preço do álcool é até 70% do preço da gasolina, se sim, ela abastece o carro com álcool, senão ela abastece o carro com gasolina. Você pode ajudar a Márcia também?

Análise

- Quais informações são relevantes e quais podem ser descartadas?
- Existe alguma omissão?
- Existe alguma ambiguidade?
- Quais conhecimentos do domínio do problema são necessários?
- O que deve ser feito?

Resultado

Determinar o combustível que será utilizado. Se o preço do álcool for até 70% do preço da gasolina, então deve-se usar o álcool, senão a gasolina.

Definição dos tipos de dados

Análise

Determinar o combustível que será utilizado. Se o preço do álcool for até 70% do preço da gasolina, então deve-se usar o álcool, senão a gasolina.

Definição dos tipos de dados

- Quais são as informações envolvidas no problema?
- Como as informações serão representadas?

Resultado

Informações: preço do litro do combustível e o tipo do combustível. Representações:

```
/// O preço do litro do combustível.  
/// Requer que seja um número positivo.
```

```
type Preco =  
    Float
```

```
/// O tipo do combustível. Requer que seja "Álcool" ou "G"  
type Combustivel =  
    String
```

A declaração `type Preço = Float` não cria um tipo novo, ela apenas dá um novo nome, um **apelido**, para um tipo que já existe.

Ou seja, `Preço` e `Float` são intercambiáveis, e o compilador não impede que um `Float` qualquer, como um preço negativo, seja usado onde esperamos um `Preço`.

Então por que usar apelidos? Porque o nome diz o que o valor representa e o comentário de documentação registra as restrições que o programa espera, mesmo que o compilador não as verifique.

Análise

Determinar o combustível que será utilizado.
Se o preço do álcool for até 70% do preço da gasolina, então deve-se usar o álcool, senão a gasolina.

Tipos

```
/// O preço do litro do combustível.  
/// Requer que seja um número positivo.  
type Preço =  
    Float  
  
/// O tipo do combustível.  
/// Requer que seja "Álcool" ou "Gasolina".  
type Combustivel =  
    String
```

Especificação

- Assinatura da função
- Propósito (o que a função faz)
- Exemplos de entrada e saída

Resultado

```
/// Encontra o combustível que deve  
/// ser utilizado no abastecimento.  
/// Produz "Álcool" se *preco_alcool*  
/// for até 70% do *preco_gasolina*,  
/// produz "Gasolina" caso contrário.  
fn seleciona_combustivel(  
    preco_alcool: Preço,  
    preco_gasolina: Preço,  
) -> Combustivel {  
    todo  
}
```

O **todo** no corpo da função é uma palavra-chave do Gleam que marca uma parte do programa que ainda não foi escrita.

Com ela a função já compila e a assinatura já é verificada, mas o `sgleam` avisa que o código está incompleto, e o programa termina com erro se a execução chegar no **todo**.

Na especificação usamos o **todo** justamente para deixar explícito que só a especificação está pronta; a implementação vem depois.

Exemplos

```
pub fn seleciona_combustivel_examples() {  
  // 3.00 é maior que 70% de 4.00, que é 2.80  
  // -> "Gasolina"  
  check.eq(seleciona_combustivel(3.0, 4.0), "Gasolina")  
  // 2.90 é menor que 70% de 4.20, que é 2.94  
  // -> "Álcool"  
  check.eq(seleciona_combustivel(2.9, 4.2), "Álcool")  
}
```

Usamos uma expressão `check.eq(exemplo, esperado)` para especificar cada exemplo e o valor esperado. O sgleam executa automaticamente todos os `check.eq` dentro de todas as funções `pub` terminadas com `_examples` e mostra um resumo dos resultados.

Falta algum exemplo?

E se o preço do álcool for exatamente 70% do preço da gasolina, como em `seleciona_combustivel(3.5, 5.0)`? Não está claro na especificação o que fazer!

Precisamos tomar uma decisão e modificar o propósito para ficar mais preciso. Quais são as possibilidades? Assumir que exatamente 70% implica no uso do álcool, assumir que implica no uso da gasolina, ou incluir uma terceira resposta, "Qualquer uma". No mundo real quem decide é a Márcia, mas aqui vamos assumir que exatamente 70% implica no uso do álcool. O propósito modificado fica

```
/// Encontra o combustível que deve ser utilizado no abastecimento. Produz  
/// "Álcool" se *preco_alcool* for menor ou igual a 70% do *preco_gasolina*,  
/// produz "Gasolina" caso contrário.  
fn seleciona_combustivel(preco_alcool: Preco, preco_gasolina: Preco) -> Combustivel {  
    todo  
}
```

Com a decisão tomada, acrescentamos o exemplo que estava faltando

```
pub fn seleciona_combustivel_examples() {  
  // 3.00 é maior que 70% de 4.00, que é 2.80  
  // -> "Gasolina"  
  check.eq(seleciona_combustivel(3.0, 4.0), "Gasolina")  
  // 2.90 é menor que 70% de 4.20, que é 2.94  
  // -> "Álcool"  
  check.eq(seleciona_combustivel(2.9, 4.2), "Álcool")  
  // 3.50 é exatamente 70% de 5.00  
  // -> "Álcool"  
  check.eq(seleciona_combustivel(3.5, 5.0), "Álcool")  
}
```

Apesar de as anotações de tipos serem opcionais, de agora em diante vamos **sempre** colocar os tipos das entradas e saídas das funções nomeadas.

No propósito da função descrevemos **o que** a função faz, e não **como** ela faz (que é a implementação — às vezes precisamos dizer como ela faz, mas isso é raro).

No propósito também informamos as garantias da saída e as restrições sobre os parâmetros.

Número par

- O que: verifica se um número é par
- Como: faz o resto da divisão do número por 2 e compara com 0; ou; faz a divisão inteira do número e multiplica por 2 e compara com o número

Ordenação

- O que: ordena os elementos de uma lista em ordem não decrescente
- Como: ordenação por seleção, por inserção, por intercalação, etc

A especificação de uma função é um **contrato** entre quem usa a função e quem a implementa.

Quem usa se compromete a respeitar as **restrições** sobre as entradas. Em troca, quem implementa se compromete a cumprir as **garantias** sobre a saída.

E se as restrições não forem respeitadas? Então o contrato não vale: a função não deve nada, ela pode produzir qualquer resposta.

Por isso as restrições e as garantias precisam estar escritas na especificação, e não apenas na cabeça de quem projetou a função.

```
/// O preço do litro do combustível.  
/// Requer que seja um número positivo.  
type Preço = Float  
/// O tipo do combustível.  
/// Requer que seja "Álcool" ou "Gasolina".  
type Combustivel = String  
/// Encontra o combustível que deve  
/// ser utilizado no abastecimento.  
/// Produz "Álcool" se *preco_alcool*  
/// for menor ou igual a 70% do  
/// *preco_gasolina*, produz "Gasolina"  
/// caso contrário.  
fn seleciona_combustivel(  
    preco_alcool: Preço,  
    preco_gasolina: Preço,  
) -> Combustivel
```

Qual é a restrição?

- Os dois preços devem ser positivos.

Qual é a garantia?

- A resposta é "Álcool" ou "Gasolina", escolhida pela regra dos 70%.

Note que parte do contrato está no propósito e parte nos comentários dos apelidos de tipos.

Uma função é **total** quando produz uma resposta válida para todos os valores dos tipos das entradas.

Uma função é **parcial** quando existem valores dos tipos das entradas para os quais ela não produz uma resposta válida. São justamente esses valores que as restrições excluem.

A `seleciona_combustivel` é total ou parcial? Parcial: o tipo das entradas é `Float`, mas um preço negativo também é um `Float`, e para ele a resposta não significa nada.

E uma função `primeiro(s: String) -> String`, que devolve o primeiro caractere de `s`? Também é parcial: para a string vazia não existe resposta.

O que acontece quando alguém chama uma função parcial com uma entrada inválida? Uma de duas coisas.

Ou a função não percebe e produz um valor sem sentido, que o resto do programa usa como se fosse uma resposta legítima. O `Arrays.binarySearch` do Java exige um vetor ordenado; se ele não estiver, a documentação diz que o resultado é indefinido: nenhum erro, só um número errado.

Ou a função percebe e interrompe o programa, como em Python ao acessar um índice fora da faixa de uma lista.

O segundo caso é melhor, porque o erro não fica escondido, mas o programa para de funcionar — e interromper o programa é um efeito colateral, justamente o que estamos evitando.

Por isso preferimos funções totais. Como transformar uma parcial em total é o assunto do capítulo **Funções totais**.

Para saber se a especificação está adequada, faça a seguinte pergunta:

Um outro desenvolvedor, que não tem acesso ao problema original e nem a análise, tem as informações necessárias na especificação para fazer uma implementação e verificação inicial?

Se a resposta for sim, então a especificação está adequada; senão, ela está incompleta.

```
fn seleciona_combustivel(  
    preco_alcool: Preco,  
    preco_gasolina: Preco,  
) -> Combustivel  
  
check.eq(  
    seleciona_combustivel(3.0, 4.0),  
    "Gasolina",  
)  
  
check.eq(  
    seleciona_combustivel(2.9, 4.2),  
    "Álcool",  
)  
  
check.eq(  
    seleciona_combustivel(3.5, 5.0),  
    "Álcool",  
)
```

Implementação

- Veremos diversas estratégias de implementação ao longo da disciplina.
- Uma delas é a **direta**. Se a forma de calcular a resposta é sempre a mesma (não depende de uma condição), então escrevemos a expressão da resposta diretamente.
- Outra é a **análise de casos**. Identificamos as formas de resposta e a condição que caracteriza cada uma delas, então escrevemos um caso para cada forma.

Temos duas formas de resposta, "Álcool" e "Gasolina", portanto, precisamos de uma condição para distinguir quando utilizar cada resposta. No caso, a resposta é "Álcool" se `preco_alcool` é menor ou igual a 70% de `preco_gasolina`; e "Gasolina" caso contrário.

```
/// Encontra o combustível que deve ser utilizado no abastecimento. Produz
/// "Álcool" se *preco_alcool* for menor ou igual a 70% do *preco_gasolina*,
/// produz "Gasolina" caso contrário.
fn seleciona_combustivel(preco_alcool: Preco, preco_gasolina: Preco) -> Combustivel {
    case preco_alcool <= 0.7 *. preco_gasolina {
        True -> "Álcool"
        False -> "Gasolina"
    }
}
```

A implementação está de acordo com a especificação? Vamos executar os exemplos e conferir.

```
import sgleam/check
/// O preço do litro do combustível. Requer que seja um número positivo.
type Preço = Float
/// O tipo do combustível. Requer que seja "Álcool" ou "Gasolina".
type Combustivel = String

fn seleciona_combustivel(preco_alcool: Preço, preco_gasolina: Preço) -> Combustivel {
  case preco_alcool <=. 0.7 *. preco_gasolina {
    True -> "Álcool"
    False -> "Gasolina"
  }
}

pub fn seleciona_combustivel_examples() {
  check.eq(seleciona_combustivel(3.0, 4.0), "Gasolina")
  check.eq(seleciona_combustivel(2.9, 4.2), "Álcool")
  check.eq(seleciona_combustivel(3.5, 5.0), "Álcool")
}
```

Para executarmos os testes, usamos o comando

No Windows

```
PS> .\sgleam test arquivo.gleam
```

No Linux ou Mac

```
$ ./sgleam test arquivo.gleam
```

A saída será algo como

```
Running tests...
```

```
3 tests, 3 success(es), 0 failure(s) and 0 error(s).
```

De forma geral, o fato de uma função produzir a resposta correta para alguns exemplos implica que a função está correta? Não!

Então por que “perder tempo” fazendo os exemplos? O primeiro objetivo dos exemplos é ajudar o projetista a entender como a saída pode ser obtida a partir das entradas. O segundo é ilustrar o seu funcionamento para que a especificação fique mais clara. Depois esses exemplos podem ser usados como uma forma inicial de verificação, que mesmo não mostrando que a função funciona corretamente, aumenta a confiança do desenvolvedor de que o código está correto.

Já que os exemplos são uma verificação inicial, então temos que ampliar a verificação? Sim! De que forma? Testes de propriedades, fuzzing, etc. Para esta disciplina, vamos utilizar apenas os exemplos para fazer a verificação.

Por que um exemplo pode falhar?

- O exemplo está errado
- A implementação está errada
- O exemplo e a implementação estão errados

```
/// O preço do litro do combustível.  
/// Requer que seja um número positivo.  
type Preço = Float  
/// O tipo do combustível.  
/// Requer que seja "Álcool" ou "Gasolina".  
type Combustivel = String  
fn seleciona_combustivel(  
    preco_alcool: Preço,  
    preco_gasolina: Preço,  
) -> Combustivel {  
    case preco_alcool <=.  
        0.7 *. preco_gasolina {  
        True -> "Álcool"  
        False -> "Gasolina"  
    }  
}
```

Revisão

- Podemos melhorar a especificação e o código?
- Podemos fazer simplificações eliminando casos especiais (generalizando)?
- Podemos criar abstrações (definição de constantes e funções)?
- Podemos renomear os objetos?

Resultado

Os tipos de dados permitem representar informações inválidas.

Veremos como lidar com essa questão no capítulo **Tipos de dados**.

Mais exemples

Os próximos três exemplos completam a revisão do processo de projeto de funções.

Eles não serão discutidos em aula, estude-os em casa, refazendo cada etapa antes de olhar a resposta.

Cada um deles mostra alguma coisa que o exemplo do combustível não mostrou: como lidar com mais de duas formas de resposta, como proceder quando o problema exige conhecimento de outra área, e como refinar o propósito a partir dos exemplos.

Exemplo - aumento de salário

O governo deu um aumento de salário para os funcionários públicos. O percentual de aumento depende do valor do salário atual. Para funcionários que ganham até R\$ 1200 o aumento é de 10%, para funcionários que ganham entre R\$ 1200 e R\$ 3000 o aumento é de 7%, para funcionários que ganham entre R\$ 3000 e R\$ 8000, o aumento é de 3%, e finalmente, para os funcionários que ganham mais que R\$ 8000 não haverá aumento. Projete uma função para calcular o novo salário de um funcionário qualquer.

Qual é o primeiro passo?

Análise

Calcular o novo salário a partir do salário atual. Se o salário for até R\$ 1200, aumento de 10%; se for maior que R\$ 1200 e até R\$ 3000, aumento de 7%; se for maior que R\$ 3000 e até R\$ 8000, aumento de 3%; se for maior que R\$ 8000, sem aumento.

Exemplo - aumento de salário

Definição de tipos de dados

```
/// Salário é um valor em reais. Requer que seja um número positivo.  
type Salariao = Float
```

Especificação

```
/// Calcula o novo salário a partir de *salario_atual*, aplicando o percentual  
/// de aumento determinado da seguinte forma  
/// - salario_atual <= 1200, aumento de 10%  
/// - 1200 < salario_atual <= 3000, aumento de 7%  
/// - 3000 < salario_atual <= 8000, aumento de 3%  
/// - 8000 < salario_atual, sem aumento  
fn novo_salario(salario_atual: Salariao) -> Salariao {  
    todo  
}
```

Exemplo - aumento de salário

Exemplos

```
pub fn novo_salario_examples() {  
    check.eq(novo_salario(1000.0), 1100.0)  
}
```

Falta alguma coisa nesse exemplo específico? Sim! Além do resultado esperado, é interessante fazer um comentário sobre como o resultado foi obtido. Esse comentário irá nos auxiliar na etapa de implementação.

```
// salario_atual <= 1200  
// -> 1000.0 *. 1.1  
check.eq(novo_salario(1000.0), 1100.0)  
check.eq(novo_salario(1200.0), 1320.0)
```

Note que também adicionamos um exemplo para o caso limite.

Exemplo - aumento de salário

```
// 1200 < salario_atual <= 3000
// -> 2000.0 *. 1.07
check.eq(novo_salario(2000.0), 2140.0)
check.eq(novo_salario(3000.0), 3210.0)

// 3000 < salario_atual <= 8000
// -> 5000.0 *. 1.03
check.eq(novo_salario(5000.0), 5150.0)
check.eq(novo_salario(8000.0), 8240.0)

// 8000 < salario_atual
// -> 8000.01
check.eq(novo_salario(8000.01), 8000.01)
```

Implementação

Quantas formas distintas de produzir o resultado da função identificamos nos exemplos?

Quatro formas (veja os comentários dos exemplos).

Como existe mais de uma forma de resposta, então usamos análise de casos. Quantos casos vamos precisar? Como são quatro formas de resposta, então precisamos de quatro casos, um para cada forma.

Em Gleam, escrevemos os quatro casos com **case** aninhados: o primeiro separa a primeira forma das demais, o segundo separa a segunda das restantes, e assim por diante.

Exemplo - aumento de salário

Com essas informações já conseguimos esboçar o corpo da função

```
fn novo_salario(salario_atual: Salarío) -> Salarío {  
  case condicao1 {  
    True -> resposta1  
    False ->  
      case condicao2 {  
        True -> resposta2  
        False ->  
          case condicao3 {  
            True -> resposta3  
            False -> resposta4  
          }  
        }  
      }  
  }  
}
```

Exemplo - aumento de salário

Agora olhamos para os exemplos, identificamos as condições que caracterizam cada caso, e fazemos a implementação. Qual a condição que caracteriza o primeiro caso?

`salario_atual <= 1200.0` E qual é a forma da resposta para esse caso?

`salario_atual *. 1.1` Com isso já podemos preencher o primeiro caso

```
fn novo_salario(salario_atual: Salarior) -> Salarior {  
  case salario_atual <= 1200.0 {  
    True -> salario_atual *. 1.1  
    False ->  
      case condicao2 {  
        True -> resposta2  
        False ->  
          case condicao3 {  
            True -> resposta3  
            False -> resposta4  
          }  
        }  
      }  
    }  
  }  
}
```

Exemplo - aumento de salário

Repetindo esse processo para os demais casos chegamos na seguinte implementação

```
fn novo_salario(salario_atual: Salarío) -> Salarío {  
  case salario_atual <=. 1200.0 {  
    True -> salario_atual *. 1.1  
    False ->  
      case salario_atual <=. 3000.0 {  
        True -> salario_atual *. 1.07  
        False ->  
          case salario_atual <=. 8000.0 {  
            True -> salario_atual *. 1.03  
            False -> salario_atual  
          }  
        }  
      }  
  }  
}
```

Note que as condições dos casos seguintes não repetem o que já foi descartado: se chegamos no segundo **case**, já sabemos que o salário é maior que 1200.

Verificação

Running tests...

7 tests, 7 success(es), 0 failure(s) and 0 error(s).

Revisão

O que podemos melhorar?

- Adicionar comentários sobre os números “mágicos”

Exemplo - aumento de salário

```
fn novo_salario(salario_atual: Salarío) -> Salarío {  
  case salario_atual <=. 1200.0 {  
    // 10% de aumento  
    True -> salario_atual *. 1.1  
    False ->  
      case salario_atual <=. 3000.0 {  
        // 7% de aumento  
        True -> salario_atual *. 1.07  
        False ->  
          case salario_atual <=. 8000.0 {  
            // 3% de aumento  
            True -> salario_atual *. 1.03  
            // sem aumento  
            False -> salario_atual  
          }  
        }  
      }  
  }  
}
```

Não podemos esquecer de fazer a verificação novamente!

O Jorge precisa determinar a massa de diversos pequenos tubos de ferro mas está sem uma balança. No entanto, ele possui um paquímetro e pode medir com precisão o diâmetro interno e externo e a altura dos tubos, agora ele só precisa de um programa para fazer os cálculos.

Algum voluntário?

Alguma coisa parece complicada nesse exercício?

Nesse exercício precisamos de conhecimento de um domínio (área), que talvez ainda não tenhamos, isso pode fazer o problema parecer mais difícil do que realmente é. Mas então, como proceder nesses casos?

Precisamos de uma pessoa (ou livros) que possam nos instruir sobre o conhecimento do domínio, geralmente os interessados no software podem indicar tais pessoas.

O importante é entender que os desenvolvedores de softwares geralmente resolvem problemas de outras pessoas, e esses problemas podem envolver conhecimentos que não temos e por isso precisamos estar dispostos a estudar e aprender o conhecimento de outras áreas.

Vamos resolver esse problema? Por onde começamos?

Análise

- Calcular a massa de um tubo de ferro a partir das suas dimensões. Como as dimensões de um tubo de ferro estão relacionadas com a massa do tubo?
- Dimensões $\rightarrow$ Volume $\rightarrow$ Massa
- Como determinamos o volume de um tubo de ferro a partir das suas dimensões?

$$volume = \pi \times (raio_externo^2 - raio_interno^2) \times altura$$

$$\text{onde } raio_externo = \frac{dimetro_externo}{2} \text{ e } raio_interno = \frac{dimetro_interno}{2}$$

- Como obtemos a massa a partir do volume? A massa é dada por $volume \times densidade$.
- Qual a densidade do ferro? A densidade do ferro é 7874 kg/m^3 .

Exemplo - massa tubo de ferro

Definição de tipos de dados

```
/// Comprimento é um número positivo dado em metros.
```

```
type Comprimento = Float
```

```
/// Massa é um número positivo dado em quilogramas.
```

```
type Massa = Float
```

Especificação

```
/// Calcula a massa de um tubo de ferro a partir das suas dimensões.
```

```
/// Requer que *diâmetro_externo* seja maior que *diâmetro_interno*.
```

```
fn massa_tubo_ferro(  
  diâmetro_externo: Comprimento,  
  diâmetro_interno: Comprimento,  
  altura: Comprimento,  
) -> Massa {  
  todo  
}
```

Exemplos

```
pub fn massa_tubo_ferro_examples() {  
  // -> 3.14 *. { 0.025 *. 0.025 -. 0.015 *. 0.015 } *. 0.1 *. 7874.0  
  check.eq(massa_tubo_ferro(0.05, 0.03, 0.1), 0.9889744)  
}
```

Implementação

Precisamos utilizar análise de casos na implementação? Não! Por quê? Porque só existe uma forma de resposta, ou seja, a resposta é sempre calculada com a mesma expressão. Essa é a implementação **direta**. E é por isso também que um único exemplo já foi suficiente.

E que expressão é essa? A que identificamos na análise do problema e utilizamos para calcular a resposta do exemplo.

Exemplo - massa tubo de ferro

```
fn massa_tubo_ferro(  
    diametro_externo: Comprimento,  
    diametro_interno: Comprimento,  
    altura: Comprimento,  
) -> Massa {  
    let raio_externo = diametro_externo /. 2.0  
    let raio_interno = diametro_interno /. 2.0  
    3.14  
    *. { raio_externo *. raio_externo -. raio_interno *. raio_interno }  
    *. altura  
    *. 7874.0  
}
```

Verificação

A verificação falha!

```
Failure at massa_tubo_ferro.gleam (massa_tubo_ferro_examples:29)
```

```
  Actual   : 0.98897440000000004
```

```
  Expected: 0.9889744
```

Comparação de igualdade de números de ponto flutuante quase nunca dá certo! Nesses casos usamos `check.approx`, que permite especificar uma margem de erro.

```
pub fn massa_tubo_ferro_examples() {  
  check.approx(massa_tubo_ferro(0.05, 0.03, 0.1), 0.9889744, 0.00000001)  
}
```

Revisão

O que podemos melhorar?

- Definir constantes para os números “mágicos”
- Separar o cálculo do volume em etapas, criando uma função para a área do círculo

Exemplo - massa tubo de ferro

```
// Na prática precisamos de mais casas decimais!
const pi: Float = 3.14
// Densidade do ferro, em kg/m^3.
const densidade_ferro: Float = 7874.0

fn area_circulo(raio: Comprimento) -> Float {
  pi *. raio *. raio
}

fn massa_tubo_ferro(
  diametro_externo: Comprimento,
  diametro_interno: Comprimento,
  altura: Comprimento,
) -> Massa {
  let area_externa = area_circulo(diametro_externo /. 2.0)
  let area_interna = area_circulo(diametro_interno /. 2.0)
  let volume = { area_externa -. area_interna } *. altura
  volume *. densidade_ferro
}
```

Em um determinado programa é preciso exibir textos em uma quantidade máxima de espaço (número de caracteres). Se o texto não cabe no espaço, apenas a parte inicial do texto que cabe no espaço junto de três pontos deve ser exibida. Além disso, o texto pode ser alinhado a direita, a esquerda ou centralizado. Projete uma função que transforme um texto para que possa ser exibido no espaço desejado.

Análise

Ajustar um texto a um tamanho específico, usando "...", se necessário, para sinalizar que o texto foi abreviado, e alinhar o texto a direita, a esquerda ou no centro.

Definição de tipos de dados

```
/// Alinhamento é um dos valores "direita", "esquerda" ou "centro".  
type Alinhamento = String
```

Especificação

```
/// Produz uma nova string a partir de *s* que tem exatamente *num_chars*  
/// caracteres e é alinhada de acordo com o *alinhamento*.  
fn ajusta_string(s: String, num_chars: Int, alinhamento: Alinhamento) -> String {  
    todo  
}
```

Essa especificação é precisa o bastante para fazermos uma implementação ou para usarmos essa função? Não.

```
pub fn ajusta_string_examples() {  
    check.eq(ajusta_string("casa", 4, "direita"), "casa")  
    check.eq(ajusta_string("casa", 4, "esquerda"), "casa")  
    check.eq(ajusta_string("casa", 4, "centro"), "casa")  
    check.eq(ajusta_string("casa verde", 7, "direita"), "casa...")  
    check.eq(ajusta_string("casa verde", 7, "esquerda"), "casa...")  
    check.eq(ajusta_string("casa verde", 7, "centro"), "casa...")  
    check.eq(ajusta_string("casa verde", 9, "direita"), "casa v...")  
    check.eq(ajusta_string("casa", 9, "direita"), "      casa")  
    check.eq(ajusta_string("casa", 9, "esquerda"), "casa      ")  
    check.eq(ajusta_string("casa", 9, "centro"), "   casa   ")  
    check.eq(ajusta_string("casa", 10, "centro"), "    casa    ")  
}
```


O que está faltando nos exemplos?

A forma como as saídas foram computadas e as respectivas condições!

Lembrem-se, o objetivo inicial dos exemplos é ajudar o projetista a entender como a função deve funcionar.

Exemplo - ajuste de texto

```
// string.length(s) == num_chars
// -> s
check.eq(ajusta_string("casa", 4, "direita"), "casa")
check.eq(ajusta_string("casa", 4, "esquerda"), "casa")
check.eq(ajusta_string("casa", 4, "centro"), "casa")

// string.length(s) > num_chars
// -> string.slice("casa verde", 0, 7 - 3) <> "..."
check.eq(ajusta_string("casa verde", 7, "direita"), "casa...")
check.eq(ajusta_string("casa verde", 7, "esquerda"), "casa...")
check.eq(ajusta_string("casa verde", 7, "centro"), "casa...")
check.eq(ajusta_string("casa verde", 9, "direita"), "casa v...")
```

```
// string.length(s) < num_chars && alinhamento == "direita"
// -> string.repeat(" ", 9 - string.length("casa")) <> "casa"
check.eq(ajusta_string("casa", 9, "direita"), "      casa")

// string.length(s) < num_chars && alinhamento == "esquerda"
// -> "casa" <> string.repeat(" ", 9 - string.length("casa"))
check.eq(ajusta_string("casa", 9, "esquerda"), "casa      ")
```

```
// string.length(s) < num_chars && alinhamento == "centro"
// -> string.repeat(" ", num_espacos_inicio)
//    <> "casa"
//    <> string.repeat(" ", num_espacos_fim)
// onde
//   num_espacos_inicio é { 9 - string.length("casa") } / 2
//   num_espacos_fim é 9 - string.length("casa") - num_espacos_inicio
check.eq(ajusta_string("casa", 9, "centro"), "  casa  ")
check.eq(ajusta_string("casa", 10, "centro"), "   casa  ")
```

Detalhamento do propósito da função a partir do aprimoramento do nosso entendimento obtido com os exemplos.

```
/// Produz uma nova string a partir de *s* que tem exatamente *num_chars*  
/// caracteres e é alinhada de acordo com o *alinhamento*.  
///  
/// Se *s* tem exatamente *num_chars* caracteres, então produz *s*.  
///  
/// Se *s* tem mais do que *num_chars* caracteres, então *s* é truncada e "..."  
/// é adicionado ao final para sinalizar que a string foi abreviada.  
///  
/// Se *s* tem menos do que *num_chars* caracteres, então espaços são  
/// adicionados no início se *alinhamento* é "direita", no fim se *alinhamento*  
/// é "esquerda", ou no início e fim se *alinhamento* é "centro". Nesse último  
/// caso, se a quantidade de espaços adicionados for ímpar, então no fim será  
/// adicionado 1 espaço a mais do que no início.
```

```
fn ajusta_string(s: String, num_chars: Int, alinhamento: Alinhamento) -> String {  
  case string.length(s) == num_chars {  
    True -> s  
    False ->  
      case string.length(s) > num_chars {  
        True -> string.slice(s, 0, num_chars - 3) <> "..."  
        False -> {  
          let num_espacos = num_chars - string.length(s)  
          case alinhamento {  
            "direita" -> string.repeat(" ", num_espacos) <> s  
            "esquerda" -> s <> string.repeat(" ", num_espacos)  
            _ -> {  
              let num_espacos_inicio = num_espacos / 2  
              let num_espacos_fim = num_espacos - num_espacos_inicio  
              string.repeat(" ", num_espacos_inicio)  
              <> s  
              <> string.repeat(" ", num_espacos_fim)  
            }  
          }  
        }  
      }  
  }  
}
```

No **case** sobre o **alinhamento** a expressão examinada é uma **String**, e não um **Bool**.

No capítulo **Fundamentos** vimos a forma inicial do **case**, com os casos **True** e **False**; ela é um caso particular. A expressão examinada pode ser de qualquer tipo, e cada caso pode ser um literal desse tipo.

```
> case 2 + 1 {  
  1 -> "um"  
  2 -> "dois"  
  _ -> "muitos"  
}  
"muitos"
```

A regra de avaliação é a mesma: avalie a expressão examinada e substitua toda a expressão **case** pela expressão do **primeiro** caso que corresponder ao valor.

O último caso do `case` sobre o `alinhamento` é `_` e não `"centro"`.

O `_` corresponde a qualquer valor, isto é, esse caso contempla todos os valores que não foram tratados nos casos anteriores; por isso ele é sempre o último caso.

Ele é necessário porque `Alinhamento` é apenas um apelido de `String`, então além de `"direita"`, `"esquerda"` e `"centro"` existem infinitos outros valores possíveis, e o Gleam exige que todos eles sejam tratados.

Veremos como evitar essa situação no capítulo **Tipos de dados**.

Implementação, versão alternativa

```
/// Produz uma nova string a partir de *s* que tem exatamente *num_chars*  
/// caracteres e é alinhada de acordo com o *alinhamento*.
```

Na especificação podemos notar um “e”, indicando que a função faz duas coisas. Então, podemos implementar a função decompondo ela nessas “duas coisas”. Supomos que as funções existem e implementamos o corpo

```
fn ajusta_string(s: String, num_chars: Int, alinhamento: Alinhamento) -> String {  
    alinha(limita(s, num_chars), num_chars, alinhamento)  
}
```

Exemplo - ajuste de texto

Agora colocamos essas duas funções em uma **lista de trabalho** (lista de desejos), isto é, a lista de funções que ainda precisam ser projetadas, cada uma com uma especificação inicial, e depois procedemos para implementá-las seguindo as mesmas etapas

```
/// Produz uma nova string a partir de *s* com no máximo *num_chars*  
/// caracteres.
```

```
/// ...
```

```
fn limita(s: String, num_chars: Int) -> String {  
    todo  
}
```

```
/// Produz uma nova string a partir de *s* alinhada de acordo com o  
/// *alinhamento*.
```

```
/// ...
```

```
fn alinha(s: String, num_chars: Int, alinhamento: Alinhamento) -> String {  
    todo  
}
```

Verificação (da primeira implementação)

- Ok

Revisão

- Exercício para o leitor!

Revisão

Quais são as etapas do processo de projeto de funções e o objetivo de cada uma?

- Análise (identificar o problema), definição dos tipos de dados (representar as informações), especificação (dizer com precisão o que a função faz), implementação, verificação (conferir a implementação contra a especificação) e revisão (melhorar o que foi feito).

Por que não devemos ir direto para a implementação?

- Porque cada etapa depende das anteriores: sem a análise não sabemos qual é o problema e sem a especificação não temos como saber se a implementação está certa.

Como sabemos se uma especificação está adequada?

- Se um outro desenvolvedor, sem acesso ao problema original e nem à análise, consegue fazer a implementação e a verificação inicial só com ela. Senão, ela está incompleta.

No propósito de uma função descrevemos o que ela faz ou como ela faz?

- O que ela faz; o como é a implementação. Por exemplo, “verifica se um número é par” é o que, “calcula o resto da divisão por 2 e compara com 0” é o como. No propósito também informamos as garantias da saída e as restrições sobre os parâmetros.

Para que servem os exemplos? Passar em todos eles mostra que a função está correta?

- Não mostra. Eles servem primeiro para o projetista entender como a saída é obtida a partir da entrada, depois para deixar a especificação mais clara e, por fim, como verificação inicial, que aumenta a confiança de que o código está correto sem provar que ele está.

O que é o contrato de uma função?

- É o acordo registrado na especificação: quem chama se compromete a respeitar as restrições sobre as entradas e, em troca, quem implementa se compromete a cumprir as garantias sobre a saída.

Qual é a diferença entre uma função total e uma função parcial?

- A total produz uma resposta válida para todos os valores dos tipos das entradas; a parcial não, e as restrições excluem justamente os valores para os quais ela não produz.

O que pode acontecer quando uma função parcial é chamada com uma entrada inválida?

- Ou ela produz um valor sem sentido, que o resto do programa usa como resposta legítima, ou ela interrompe o programa.

Quais estratégias de implementação vimos e quando usar cada uma?

- A direta, quando existe uma única forma de calcular a resposta, e a análise de casos, quando existe mais de uma, com um caso para cada forma.

Quando um exemplo falha, o que pode estar errado?

- O exemplo, a implementação, ou os dois.

Referências

Básicas

- Capítulo 3 - How to Design Programs do livro HTDP
- Vídeos BSL
- Vídeos How to Design Functions